

ASSIGMENT 3 []
SANTIAGO VILLARREAL VILLARRAGA
8043768
VILLARRS@MYUMANITOBA.CA
01-03-2025

CONTENTS

1. Problems	1
2. Additional Discussion Questions	5
References	6

I, Santiago Villarreal attest that the work I am submitting is my own work and that it has not been copied/plagiarized from online or other sources. Any sourced material used for completing this work has been properly cited. Santiago Villarreal Villarraga

1. PROBLEMS

P1-1 Train and evaluate both models for the same choice of hyperparameters. Comment on training performance and support your arguments with training curve plots

I decided to use the data set of fashion misnt for faster results, I have to change the code for the DC-GAN, changing the input of the discriminator and the output of the generator to 28x28x1, for the first time i had to try different combinations of hiperparameter to have the DC-GAN stable since the loss of the generator was getting stuck in a high loss (over 15).

```
dcgan.compile(  
    d_optimizer=keras.optimizers.Adam(learning_rate=0.00001),  
    g_optimizer=keras.optimizers.Adam(learning_rate=0.00003),  
    loss_fn=keras.losses.BinaryCrossentropy(),  
)
```

Thanks to the previous testing, I found that is easier to train a WGAN-GP.

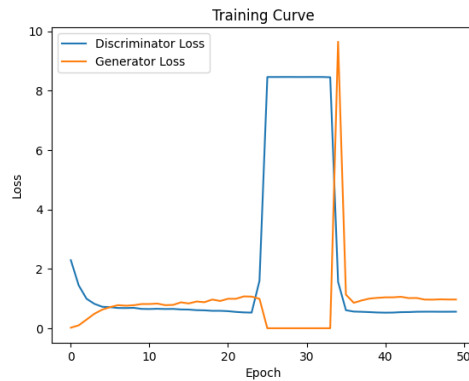


FIGURE 1. Training DC-GAN

As we can see in the previous plot, the training of the DC GAN tends to get stuck in some values. In this case, it was able to recover, but there were some cases when the training rate was higher that it was stuck in high losses for both the discriminator and the generator. The next plot is for the WGAN.

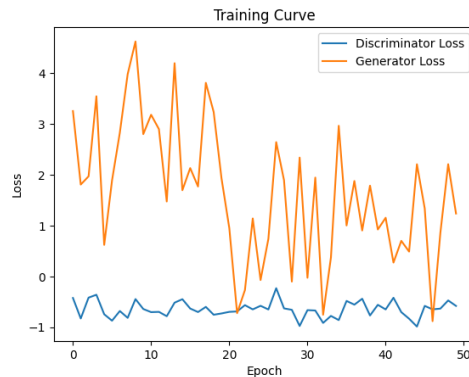


FIGURE 2. Training WGAN

In the previous plot, the training in the WGAN is more stable and allows larger learning rates, also we can see with more detail the nature of the GAN, where the generator loss goes up and down.

P1-2 For either DCGAN or WGAN-GP (or both if you are feeling adventurous) change a hyperparameter (your choice) and comment on the effect that it has on training. Is the training sensitive or not? Justify your answer.

```
dcgan.compile(
    d_optimizer=keras.optimizers.Adam(learning_rate=0.0001),
    g_optimizer=keras.optimizers.Adam(learning_rate=0.0001),
    loss_fn=keras.losses.BinaryCrossentropy(),
)
```

I decide to use the DCGAN to show how unstable is it, i changed the learning rate to 0.001 and re train it, as we can see in the next plot, the loss

is stuck, the generator loss is near 0, I would say is that the generator is generating few images that allows to fool the discriminator.

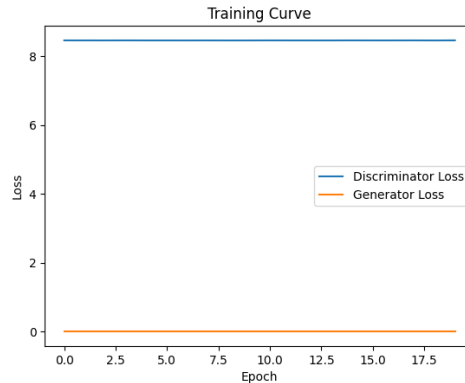


FIGURE 3. Larger learning rate

P2-1 Given the Conditional GAN notebook (Lecture 10 Program a) explain, at a high level, how the conditional GAN is implemented, focusing on the loss function and how attributes are handled. What flavour of Conditional GAN is this?

The Conditional GAN is designed to generate handwritten digits based on a class label. The generator combines random noise with one-hot encoded labels to create images that match the specified class. The discriminator, on the other hand, takes in both real or generated images along with their corresponding labels to determine if the images are real or fake. The model uses Binary Crossentropy loss: the discriminator learns to correctly classify real images while identifying fake ones, and the generator tries to fool the discriminator into thinking its generated images are real. During training, real images with labels and generated images with labels are passed through the discriminator to update its accuracy, while the generator continuously improves by using the discriminator's feedback. This CGAN uses a simple concatenation method, where labels are attached to the input noise for the generator and to the images for the discriminator, helping the model produce images that correspond to the given labels.

P3-1 Explain how the loss function is implemented, including what terms are evaluated, why they are included, and how they are weighted. Note that there is a discussion of the loss function in Lecture 10 that was not covered in detail in class.

The CycleGAN loss function consists of three main components: adversarial loss, cycle consistency loss, and identity loss, all of which are computed for both generators (G and F) and discriminators (X and Y) during training. The adversarial loss ensures the generators create realistic images that fool the discriminators, implemented as the Mean Squared Error between the discriminator's output for fake images and a target value of 1. The cycle consistency loss maintains translation consistency between domains, measured using the Mean Absolute Error between original and cycled images, weighted

by a hyperparameter λ cycle. The identity loss encourages generators to preserve input images when translating within the same domain, also calculated with Mean Absolute Error and influenced by both λ cycle and λ identity. The total generator loss sums these three components, while the discriminator loss separately evaluates real and fake images using Mean Squared Error to ensure correct classification. The weighting of loss terms via λ cycle and λ identity is crucial for balancing these losses and optimizing image translation performance.

P3-2 Explain the training process, focusing on the relationship between generators and discriminators.

The CycleGAN training process is a continuous competition between two generators and two discriminators. The generators are responsible for translating images between two domains, such as horses to zebras and vice versa, while the discriminators distinguish between real and generated images in each domain. The training alternates between updating the generators, which aim to create realistic images while preserving features through cycle consistency and identity mapping, and updating the discriminators, which learn to differentiate real from fake images. The generators' losses are based on how well they fool the discriminators, how closely translated images resemble their originals after being cycled back, and how well they preserve key features. The discriminators are trained to correctly classify real and fake images, constantly adjusting their weights to improve accuracy. This adversarial process forces the generators to improve at fooling the discriminators while the discriminators become more skilled at detecting fakes, creating a balanced competition that ultimately results in high-quality image translations.

P3-3 Augment the code to monitor each component of the loss function and plot the loss function through training. Which loss term(s) dominate? Are there general trends? If so can you justify why?

As we can see in the figure 5, The loss for both generators dominates the overall loss, as they are mainly responsible for producing realistic images and ensuring consistency. After that, we observe a downward trend in the generator losses, indicating that the generators are improving in producing more convincing fake images that fool the discriminators and maintaining consistency. The identity loss (id loss) also tends to decrease as the generators better preserve the characteristics of images from the target domain. But, the discriminator losses are generally smaller because they quickly learn to distinguish between real and fake images. However, as the generators improve, the discriminator losses may increase slightly as the discriminators struggle to keep up with the increasingly realistic outputs from the generators.

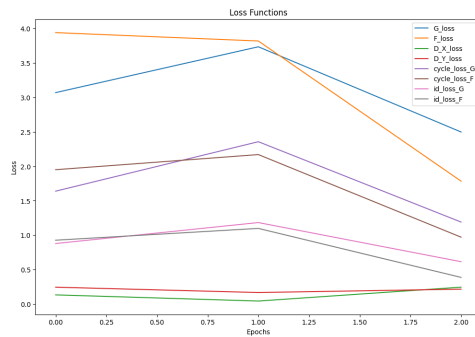


FIGURE 4. Loss plot

P3-4 Modify the loss function by dropping a term (your choice). Before running the code, predict what effect it will have on performance (consider both sides of the cycle). Run the code, plot the loss functions throughout training, and comment on the trends and final results.

I will drop the ID G loss, dropping this loss would result in the model producing less realistic and consistent images in the target Y, as the generator would no longer be penalized for altering images that are already in Y. This could lead to a less stable training process. Additionally, the overall convergence might slow down, and the model could struggle to generalize well.

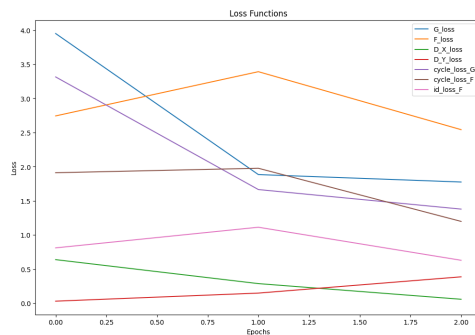


FIGURE 5. ID G dropped

as we can see the losses had a decreasing trend, changing faster than the original, but outside the plot, the images that the model was generating had less quality and a lot more noise.

2. ADDITIONAL DISCUSSION QUESTIONS

- In class we discussed InfoGAN and what it means to randomly generate inputs and attributes. For the sake of solidifying our understanding, write a short (300-word max) explanation of why InfoGAN should/could work to give meaningful information about sample features. You may want to refer to Prince or the original paper for insight.

The InfoGAN works not just generating random inputs, but also specific attributes that can control features in the generated samples. This is achieved by using a part of the input to the generator, called the latent code, which is designed to capture influential variations in the data. By maximizing the information between this latent code and the generated output, InfoGAN ensures that changes in the latent code lead to predictable and interpretable changes in the generated samples. This means that the model learns to associate specific aspects of the generated images with the latent code, which allows us to understand and manipulate these features directly. This approach helps to create more understandable and controllable output, providing more precise insights into the structure and characteristics of the data.

REFERENCES

- [1] Prince, S. (2023). Understanding Deep Learning, MIT Press.
<https://udlbook.github.io/udlbook/>
- [2] Foster, D. (2023). Generative deep learning, 2nd Edition. O'Reilly Media, Inc.
ISBN:9781492041948
- [3] Murphy, K. P. (2022). Probabilistic machine learning: An introduction. MIT Press.
<https://probml.github.io/pml-book/>
- [4] Murphy, K. P. (2023). Probabilistic machine learning: Advanced topics. MIT Press.
<https://probml.github.io/pml-book/>